Binary Search Tree and Red Black Tree

Author: Eric Zhou
Email: eric_ds_zhou@outlook.com
Date: Oct 20, 2026

Introduction

This technical note summarizes the tools and logic used in Binary Search Tree and Red Black Tree

Table of contents

- What Is a Binary Search Tree?
- Basic Concepts
  - Sibling
  - Successor
  - Predecessor
  - Height
  - Minimum and Maximum Values
- Operations in a Binary Search Tree
  - Search
  - Insertion
  - Deletion
- What Is a Balanced BST?
- Rotations in a BST
  - Left Rotation
  - Right Rotation
- Advantages and Limitations of BSTs
  - Time Complexity of BST Operations
  - Balanced vs. Unbalanced BSTs
  - BST vs. Red-Black Tree: Time Complexity Comparison
- What Is a Red-Black Tree?
- Properties of a Valid Red-Black Tree
- Insertion in a Red-Black Tree
  - Case I: Parent and Uncle Are Both Red
    - Recoloring
  - Case II: Parent Is Red and Uncle Is Black
    - LR Case
    - RL Case
  - Case III: Parent Is Red and Uncle Is Black
    - LL Case
    - RR Case
  - Root Recoloring
- Practice Problem: BST Left Rotation and Balance Update (15 Points)
- Reference List

What Is a Binary Search Tree?

A **Binary Search Tree (BST)** is a binary tree that satisfies the **Binary Search Tree Property**.

A binary tree is a tree with the following properties:

1. Every node has zero, one, or two children.
2. One node is designated as the **root**.
3. Every node except the root has exactly one parent.
4. A node with no children is called a **leaf**.

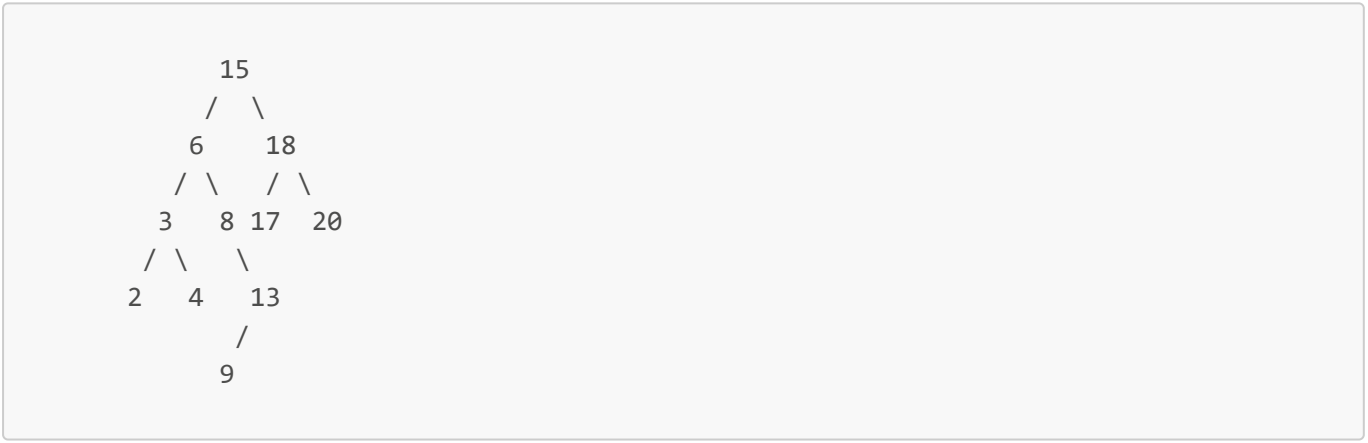
In a BST, for any node `x`:

- Every node `y` in the left subtree of `x` satisfies `y.key <= x.key`.
- Every node `y` in the right subtree of `x` satisfies `y.key >= x.key`.

This property must hold for **every node** in the tree, not just the root.

Example

Consider the following Binary Search Tree:



For example:

- All values in the left subtree of `15` are less than or equal to `15`.
- All values in the right subtree of `15` are greater than or equal to `15`.
- The node `13` is in the right subtree of `8`, since `13 > 8`.

An important property of a BST is that an **in-order traversal** visits all nodes in non-decreasing order.

For this tree, the in-order traversal produces:

```
2, 3, 4, 6, 8, 9, 13, 15, 17, 18, 20
```

The running time of an in-order traversal is:

$$T(n) = \Theta(n)$$

because every node is visited exactly once.

Practice Problem: BST Left Rotation and Balance Update (15 Points)

Consider a Binary Search Tree (BST) in which each node `v` stores the following information:

- `v.left`: Pointer to the left child
- `v.right`: Pointer to the right child
- `v.parent`: Pointer to the parent
- `v.height`: Height of the subtree rooted at `v`
- `v.balance`: Absolute difference between the heights of its left and right subtrees

The balance of a node is defined as:

$$v.balance = |v.left.height - v.right.height|$$

The height of a node is defined as:

$$v.height = 1 + \max(v.left.height, v.right.height)$$

Assume `NIL.height = 0`.

Given Function

You are given the following function:

`LeftRotate(T, z)`

This function performs a standard left rotation at node `z` and correctly updates all parent and child pointers. However, it **does not update any height or balance values**.

Task

Write pseudocode for a function:

`LeftRotateAndUpdateBalance(T, z)`

Your function must:

1. Perform a left rotation at node `z`.
2. Correctly update the `height` and `balance` of all potentially affected nodes.
3. Avoid unnecessarily traversing the entire tree.
4. Assume `z.right != NIL`.

You may use `max(a, b)` and `abs(a)`.

Complexity Analysis

Analyze the **worst-case time complexity** of your algorithm in terms of the height h of the tree.

Solution

```
LeftRotateAndUpdateBalance(T, z) {
    LeftRotate(T, z)

    while (z != NIL) {
```

```
            z.height = 1 + max(z.left.height, z.right.height)
            z.balance = |z.left.height - z.right.height|
            z = z.parent()
        }
    }
```

Worst-case Time Complexity:

$$\boxed{\Theta(h)}$$

After the left rotation, the algorithm updates the height and balance of each node along the path from the original z to the root. Each update takes constant time, and there are at most $h + 1$ nodes on this path.

Therefore, the worst-case time complexity is $\Theta(h)$.

Reference List

1. "*CSE2331_Midterm_4_Review*" Created by Professor Painter.
2. "*CSE2331_Binary_Search_Tree_Homework*" Created by Professor Painter.
3. "*CSE2331_Binary_Search_Tree_Outline*" Created by Professor Painter.